

Scrum Practice — Progetto Music Library

Data: 22/06/2026

Membri del gruppo: Schirò Valentina Carmen

Step 1: Preparazione

Trello Board:

<https://trello.com/invite/b/6a2ff6b7a12185a957b4cd80/ATTladec8b6ac10c98bebb1225d00c61f6f5ACEDC717/music-library>

User stories prodotte in questa fase:

Epic H1 — Track & Playlist Management + Basic Playback:

- US-H1.1 Creating a track
- US-H1.2 Editing a track
- US-H1.3 Deleting a track
- US-H1.4 View song list
- US-H1.5 Creating a playlist
- US-H1.6 Adding tracks to a playlist
- US-H1.7 Removing a track from a playlist
- US-H1.8 Playing a single track
- US-H1.9 Play an entire playlist
- US-H1.10 Pause and resume playback
- US-H1.11 Navigate tracks

Epic H2 — Playback Modes + Edit during play:

- US-H2.1 Sequential mode
- US-H2.2 Shuffle mode
- US-H2.3 Loop mode
- US-H2.4 Adding or removing tracks during playback

Epic H3 — Real-time MediaPlayer UI:

- US-H3.1 Real-time playback in Media Player

Epic M1 — Undo + Home Page + Auto Playlist Genre/Year:

- US-M1.1 Undo adding/removing operations
- US-M1.2 Most-played tracks on the homepage
- US-M1.3 Most-played playlists on the Home Page
- US-M1.4 Automatic creation of playlists by genre
- US-M1.5 Automatic creation of playlists by year

Epic M2 — Visual tags + Auto Playlist for Tag:

- US-M2.1 Visual tag: Favorite
- US-M2.2 Visual tag: Explicit
- US-M2.3 Visual tag: New Release
- US-M2.4 Automatic creation of playlists by tag

Epic L1 — Track Rearrangement:

- US-L1.1 Rearrange tracks in the playlist

Definition of Done:

- Il codice compila senza errori
- Tutti gli Acceptance Criteria della User Story sono soddisfatti
- Ogni AC relativo alla logica di business ha almeno un test JUnit associato

- Gli AC relativi alla UI sono verificati manualmente
- Ogni checkpoint intermedio ha almeno un test JUnit che passa
- Nessuna regressione sui test esistenti
- Il codice è documentato con Javadoc sui metodi pubblici
- Codice committato su GitHub con messaggio descrittivo

Fase 2.1: Primo Sprint — Planning

Stima iniziale della Project Velocity: 11 SP

User stories scelte per questo Sprint (totale 11 SP):

- US-H1.1 Creating a track (2 SP)
- US-H1.2 Editing a track (2 SP)
- US-H1.3 Deleting a track (3 SP)
- US-H1.4 View song list (1 SP)
- US-H1.5 Creating a playlist (1 SP)
- US-H1.6 Adding tracks to a playlist (2 SP)

Fase 2.2: Primo Sprint — Development

User stories aggiunte o modificate dal Product Owner:

A seguito del Backlog Refinement (criteri INVEST), alcuni acceptance criteria che dipendevano dalla riproduzione — non inclusa in questo Sprint — sono stati ricollocati nelle storie che la implementano:

- US-H1.2 (modificata): rimosso lo Scenario 3 «modifica del brano in riproduzione», spostato in US-H3.1
- US-H1.3 (modificata): rimossi gli Scenari 3 e 4 «eliminazione del brano in riproduzione», spostati in US-H2.4
- US-H3.1 (modificata): accoglie lo scenario di modifica del brano durante la riproduzione
- US-H2.4 (modificata): accoglie gli scenari di eliminazione del brano durante la riproduzione

Fase 2.3: Primo Sprint — Review

User stories complete al termine di questo Sprint:

- US-H1.1 Creating a track
- US-H1.2 Editing a track
- US-H1.3 Deleting a track
- US-H1.4 View song list
- US-H1.5 Creating a playlist
- US-H1.6 Adding tracks to a playlist

Project Velocity misurata: 11 story point (2 + 2 + 3 + 1 + 1 + 2).

Fase 2.4: Primo Sprint — Retrospective

Le decisioni prese nella retrospettiva di questo Sprint (diagramma a stella marina):

Stop (cose da smettere di fare):

- Scrivere acceptance criteria che dipendono da funzionalità non incluse nello Sprint

Less of (cose da ridurre):

- Duplicazione di logica tra classi (risolta estraendo metodi condivisi, es. `validateTrackData`)

Keep doing (cose da continuare a fare):

- Progettazione basata su design pattern (Facade, Command, Repository) per alta coesione e basso accoppiamento
- Sviluppo guidato dai test (TDD) con JUnit
- Integrazione frequente del codice e checkpoint intermedi che compilano e passano i test

More of (cose da fare di più):

- Boundary value testing sin dall'inizio (analisi dei valori limite)
- Commenti Javadoc contestuali sui metodi pubblici man mano che si scrive il codice

Start (cose da iniziare a fare):

- Validare gli acceptance criteria con i criteri INVEST (in particolare Independent e Testable) già durante lo Sprint Planning
- Utilizzare GitHub più spesso